Protocol Audit Report

Version 1.0

First Audit Report

April 24, 2026

Protocol Audit Report

Anamika

April 24, 2026

Prepared by: [Anamika] Lead Auditors: - Anamika

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    * [H-1] Storing the password on-chain makes it visible to anyone and no longer private
    * [H-2] `PasswordStore::setPassword` lacks access control, allowing anyone to change the password
  - Informational
    * [I-1] Incorrect NatSpec in `PasswordStore::getPassword`

Protocol Summary

PasswordStore is a protocol dedicated to storage and retrieval of a user's password. The protocol is designed to be used by a single user, and is not designed to be used by multiple users. Only the owner should be able to set and access this password.

Disclaimer

The YOUR_NAME_HERE team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

commit Hash:

```
7d55682ddc4301a7b13ae9413095feffd9924566
```

Scope

./src == PasswordStore.sol

Roles

- Owner: The user who can set the password and read the password.
- Outsides: No one else should be able to set or read the password.

Executive Summary

I took the Cyfrin Updraft `Smart Contract Security` course, and this is my first audit.

Issues found

Severity	Number of Issues Found
High	2
Medium	
Low	
Info	1
Total	3

Findings

High

[H-1] Storing the password on-chain makes it visible to anyone and no longer private

Description:
All data stored on-chain is visible to anyone and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be private and only accessed through the `PasswordStore::getPassword` function, which is intended to be callable only by the contract owner.

We demonstrate one such method of reading data off-chain below.

Impact:
Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept:
The test case below shows how anyone can read the password directly from the blockchain.

1. Create a locally running chain:

```
make anvil
```

2. Deploy the contract to the chain:

```
make deploy
```

3. Read the storage slot:
   We use slot 1, as it stores `s_password` in the contract.

```
cast storage <contract_address> 1 --rpc-url http://127.0.0.1:8545
```

You will get output similar to:

```
0x6d7950617373776f72640000000000000000000000000000000000000000000014
```

You can then parse the hex to a string:

```
cast parse-bytes32-string 0
    x6d7950617373776f72640000000000000000000000000000000000000000000014
```

Output:

```
myPassword
```

Recommended Mitigation:
The overall architecture of the contract should be reconsidered. One approach is to encrypt the password off-chain and store only the encrypted value on-chain. This would require the user to maintain a separate secret for decryption.

Additionally, consider removing the `view` function, as users may unintentionally expose sensitive data when interacting with the contract.

[H-2] `PasswordStore::setPassword` lacks access control, allowing anyone to change the password

Description:
The `PasswordStore::setPassword` function is declared as `external`, but based on the NatSpec and intended design, it should only be callable by the contract owner.

```
function setPassword(string memory newPassword) external {
    //@audit - No access control
    s_password = newPassword;
    emit SetNetPassword();
}
```

Impact:
Any user can modify the password, completely breaking the intended access control and functionality of the contract.

Proof of Concept:
Add the following test to `PasswordStore.t.sol`:

Code

```
function test_anyone_can_set_password(address randomAddress) public {
    vm.assume(randomAddress != owner);

    vm.prank(randomAddress);
    string memory expectedPassword = "myNewPassword";
    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

Recommended Mitigation:
Add access control to restrict this function to the contract owner:

```
if (msg.sender != s_owner) {
    revert PasswordStore__NotOwner();
}
```

Informational

[I-1] Incorrect NatSpec in `PasswordStore::getPassword`

Description:

```
/**
 * @notice This allows only the owner to retrieve the password
 * @param newPassword The new password to set
 */
function getPassword() external view returns (string memory) {
```

The function signature is `getPassword()` and does not accept any parameters. The NatSpec incorrectly documents a parameter (`newPassword`) that does not exist.

Impact:
Incorrect documentation can confuse developers and auditors, leading to misunderstandings about function behavior.

Recommended Mitigation:
Remove the incorrect NatSpec line:

```
- * @param newPassword The new password to set
```